



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №9

із дисципліни «Технології розробки програмного забезпечення»

**Тема: «РІЗНІ ВИДИ ВЗАЄМОДІЇ ДОДАТКІВ: CLIENT-SERVER, PEER-TO-PEER,
SERVICE-ORIENTED ARCHITECTURE»**

Виконав:
Студент групи ІА-24
Зелінський І.О.

Перевірив:
Мякий М. Ю.

Зміст

Тема.....	3
Мета.....	3
Завдання.....	3
Теоретичні відомості.....	3
Обрана тема.....	5
Реалізація функцій Java Spring Boot для CLIENT-SERVER.....	5
Реалізація технології web socket.....	12
Висновок.....	14

Тема: РІЗНІ ВИДИ ВЗАЄМОДІЇ ДОДАТКІВ: CLIENT-SERVER, PEER-TO-PEER, SERVICE-ORIENTED ARCHITECTURE

Мета: Навчитися реалізовувати різні види взаємодії додатків, а саме CLIENT-SERVER, PEER-TO-PEER, SERVICE-ORIENTED ARCHITECTURE

Завдання:

1. Ознайомитися з короткими теоретичними відомостями.
2. Реалізувати частину функціоналу робочої програми у вигляді класів і їх взаємодій для досягнення конкретних функціональних можливостей.
3. Реалізувати взаємодію програми в одній з архітектур відповідно до обраної теми.

Теоретичні відомості

Client-Server (Клієнт-Сервер)

Модель Client-Server є одним із найпоширеніших способів взаємодії додатків. У цій моделі один додаток виступає клієнтом, а інший — сервером.

Ключові характеристики:

- Централізованість: Сервер забезпечує основний функціонал та зберігає дані, тоді як клієнт лише взаємодіє з сервером для отримання або обробки інформації.
- Ролі: Сервер обробляє запити клієнта та повертає відповіді.

Приклади:

- Веб-браузери (клієнт) та веб-сервери (сервер).
- Мобільні додатки, які взаємодіють з API.

Переваги:

- Легка масштабованість серверної частини.
- Централізоване управління даними та безпекою.

Недоліки:

- Вразливість до навантаження на сервер.
- Залежність клієнта від доступності сервера.

Peer-to-Peer (P2P)

Peer-to-Peer модель взаємодії забезпечує зв'язок між рівноправними учасниками, які виконують роль як клієнта, так і сервера.

Ключові характеристики:

- Децентралізованість: Відсутність центрального сервера.
- Прямий зв'язок: Вузли мережі напряму передають дані один одному.

Приклади:

- Торрент-клієнти (BitTorrent).
- Мережі криптовалют (Bitcoin, Ethereum).

Переваги:

- Відсутність залежності від центрального сервера.
- Стійкість до збоїв окремих вузлів.

Недоліки:

- Складнощі із забезпеченням безпеки.
- Неоптимальне управління ресурсами мережі.

Service-Oriented Architecture (SOA)

SOA — це підхід до побудови додатків, де сервіси з незалежною функціональністю взаємодіють між собою через стандартизовані інтерфейси.

Ключові характеристики:

- Компонентність: Додаток розділений на окремі сервіси, кожен з яких виконує свою функцію.
- Стандарти: Взаємодія забезпечується через протоколи, такі як HTTP, SOAP або REST.

Приклади:

- Екосистеми мікросервісів.
- Корпоративні додатки на основі SOAP.

Переваги:

- Гнучкість і повторне використання сервісів.
- Незалежність технологій у різних сервісах.

Недоліки:

- Ускладнення розробки через взаємодію різних сервісів.
- Затримки у мережі при комунікації між сервісами.

Обрана тема: Online radio station (iterator, adapter, factory method, facade, visitor, client-server)

Додаток повинен служити сервером для радіостанції з можливістю мовлення на радіостанцію (64, 92, 128, 196, 224 kb/s) в потоковому режимі; вести облік підключених користувачів і статистику відвідувань і прослуховувань; налаштувати папки з піснями і можливість вести списки програвання або playlists (не відтворювати всі пісні).

Хід роботи

Під час розробки програми я реалізував взаємодію CLIENT-SERVER для свого проекту онлайн радіо станції. Spring Boot дозволяє легко реалізувати Client-Server архітектуру за допомогою REST API. Серверна частина обробляє HTTP-запити від клієнта та повертає відповіді у форматі JSON або XML.

```
package com.OnlineRadio.OnlineRadioStation.controllers;

import com.OnlineRadio.OnlineRadioStation.facade.OnlineRadioFacade;
import com.OnlineRadio.OnlineRadioStation.models.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.method.annotation.StreamingResponseBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.http.ResponseEntity;
import java.io.IOException;
import java.util.*;

@RestController
@RequestMapping("/api")
public class OnlineRadioController {
    private final Set<String> activeUsers = new HashSet<>();

    private final OnlineRadioFacade facade;

    @Autowired
    public OnlineRadioController(OnlineRadioFacade facade) {
        this.facade = facade;
    }

    @GetMapping("/admin/queue")
    @ResponseBody
    public Queue<Song> getQueue() {
        return facade.getQueue();
    }

    @PostMapping("/admin/next")
    @ResponseBody
    public void playNextSong() {
        facade.playNextSong();
    }

    @PostMapping("/admin/clear")
    @ResponseBody
    public void clearQueue() {
        facade.clearQueue();
    }

    @PostMapping("/admin/add/{songId}")
    public ResponseEntity<String> addToQueue(@PathVariable Long songId) {
        try {
```

```

        facade.addToQueue(songId);
        return ResponseEntity.ok("Song added to queue.");
    } catch (IllegalArgumentException e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    } catch (Exception e) {
        return ResponseEntity.status(500).body("An error occurred: " +
e.getMessage());
    }
}

@PostMapping("/admin/addPlaylist/{playlistId}")
public ResponseEntity<String> addPlaylistToQueue(@PathVariable Long
playlistId,

@RequestParam(defaultValue = "false") boolean shuffle) {
    try {
        facade.addPlaylistToQueue(playlistId, shuffle);
        return ResponseEntity.ok("Playlist added to queue.");
    } catch (Exception e) {
        return ResponseEntity.status(500).body("An error occurred: " +
e.getMessage());
    }
}

@GetMapping("/sync")
public Map<String, Object> syncWithRadio() {
    return facade.syncWithRadio();
}

@GetMapping("/stream")
public ResponseEntity<StreamingResponseBody> streamSong(@RequestParam
int bitrate) throws IOException {
    return facade.steamSong(bitrate);
}

@GetMapping("/history")
@ResponseBody
public ResponseEntity<Map<String, Song>> getSongHistory() {
    Map<String, Song> songHistory = facade.getFormattedSongHistory();
    return ResponseEntity.ok(songHistory);
}
}

```

Рисунок 1. Приклад контролеру OnlineRadioController

Це REST-контролер, який реалізує функціональність серверної частини для онлайн-радіо. Основні елементи:

/api/admin/queue - Отримує чергу пісень (повертає об'єкт Queue<Song>).

/api/admin/next - Перемикає відтворення на наступну пісню в черзі.

/api/admin/clear - Очищає чергу пісень.

/api/admin/add/{songId} - Додає пісню в чергу за ідентифікатором. Обробляє помилки.

/api/admin/addPlaylist/{playlistId} - Додає плейлист до черги. Може бути перемішаний.

/api/sync - Синхронізує клієнт із поточним станом відтворення (назва, артисти, час).

/api/stream?bitrate={bitrate} - Стримінг пісні у вказаній якості (бітрейті).

/api/history - Отримує історію відтворених пісень у форматі JSON.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Radio</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
<div class="header">
  <div class="header-content">
    <h2 href="/">Online Radio</h2>
    <div class="header-buttons">
      <a href="/login" class="button">Login</a>
      <a href="/register" class="button">Register</a>
      <a href="/admin" class="button admin-button">Admin Panel</a>
    </div>
  </div>
</div>
<div class="container">
  <section class="radio-controls">
    <h1>Welcome to Online Radio</h1>
    <p>Listen to your favorite music!</p>
  </section>
  <section class="radio-controls">
    <label for="bitrate">Choose bitrate:</label>
    <select id="bitrate" onchange="updateBitrate(this.value)">
      <option value="64">64 kbps</option>
      <option value="92">92 kbps</option>
      <option value="128" selected>128 kbps</option>
      <option value="196">196 kbps</option>
      <option value="224">224 kbps</option>
    </select>
    <br><br>
    <button id="playPauseButton" class="play-button"
onclick="togglePlayPause()">Play Song</button>
    <section class="audio-player">
      <audio id="audioPlayer" autoplay>
        <source type="audio/mp3">
        Your browser does not support the audio element.
      </audio>
      <p id="songTitle"></p>
      <p id="songArtists"></p>
    </section>
    <label for="volumeSlider">Volume:</label>
    <input type="range" id="volumeSlider" min="0" max="1" step="0.01"
value="1" onchange="setVolume(this.value)">
    <section class="user-info">
      <p id="userCount">Users connected: 0</p>
    </section>
  </div>
</div>
```

```

</section>
<section class="song-history">
  <h2>Song History</h2>
  <ul id="songHistoryList">
  </ul>
</section>
</div>

<div class="side-gif left">
  
</div>
<div class="side-gif right">
  
</div>
<script src="script.js"></script>
</body>
</html>

```

Рисунок 2. Приклад html сторінки

```

const socket = new WebSocket('ws://localhost:8080/ws/radio');

socket.onmessage = function(event) {
  const data = JSON.parse(event.data);
  if (data.action === "playSong") {
    if (playPauseButton.textContent === 'Pause') {
      playSong();
    }
  } else if (data.action === "updateUserCount") {
    updateUserCount(data.count);
  }
};

function updateUserCount(count) {
  const userCountElement = document.getElementById('userCount');
  userCountElement.textContent = `Users connected: ${count}`;
}

socket.onerror = function(event) {
  console.error("WebSocket error:", event);
};

socket.onclose = function(event) {
  console.log("WebSocket connection closed:", event);
};

const audioPlayer = document.getElementById('audioPlayer');
let selectedBitrate = 128;
let currentSongId = null;
updateSongHistory()

function togglePlayPause() {
  const playPauseButton = document.getElementById('playPauseButton');

  if (playPauseButton.textContent === 'Play Song') {
    playSong();
  } else if (playPauseButton.textContent === 'Pause') {
    pauseSong();
  }
}

```

```

    } else if (playPauseButton.textContent === 'Continue') {
        continueSong();
    }
}

function playSong() {
    const playPauseButton = document.getElementById('playPauseButton');

    fetch('/api/sync', {
        method: 'GET',
    })
    .then(response => {
        if (response.ok) {
            return response.json();
        } else {
            alert('Failed to synchronize with the server.');
```

throw new Error('Failed to get sync time');

```

        }
    })
    .then(syncData => {
        timeToStart = syncData.currentTime;
        currentSongId = syncData.songId;

        document.getElementById('songTitle').textContent =
`${syncData.currentTitle}`;
        document.getElementById('songArtists').textContent = `By
${syncData.currentSongArtists.join(', ')}`;

        fetch(`/api/stream?bitrate=${selectedBitrate}`, {
            method: 'GET',
        })
        .then(response => {
            if (response.ok) {
                response.blob().then(blob => {
                    const audioPlayer =
document.getElementById('audioPlayer');
                    audioPlayer.src = URL.createObjectURL(blob);

                    audioPlayer.currentTime = timeToStart;

                    audioPlayer.play();
                    playPauseButton.textContent = 'Pause';
                    updateSongHistory();
                });
            } else {
                alert('Failed to play the song.');
```

}

```

        })
        .catch(error => {
            console.error('Error:', error);
            alert('Error occurred while playing the song.');
```

});

```

    })
    .catch(error => {
        console.error('Error:', error);
    });
});

```

```

}

function pauseSong() {
  const audioPlayer = document.getElementById('audioPlayer');
  const playPauseButton = document.getElementById('playPauseButton');

  audioPlayer.pause();
  playPauseButton.textContent = 'Continue';
}

function continueSong() {
  const audioPlayer = document.getElementById('audioPlayer');
  const playPauseButton = document.getElementById('playPauseButton');

  fetch('/api/sync', {
    method: 'GET',
  })
  .then(response => {
    if (response.ok) {
      return response.json();
    } else {
      throw new Error('Failed to get sync time');
    }
  })
  .then(syncData => {
    const timeToStart = syncData.currentTime;
    const songId = syncData.songId;

    if (currentSongId !== songId) {
      currentSongId = songId;
      playSong();
    } else {
      audioPlayer.currentTime = timeToStart;
      audioPlayer.play();
      playPauseButton.textContent = 'Pause';
    }
  })
  .catch(error => {
    console.error('Error:', error);
  });
}

function updateBitrate(bitrate) {
  selectedBitrate = bitrate;
}

function setVolume(volume) {
  var audioPlayer = document.getElementById('audioPlayer');
  audioPlayer.volume = volume;
}

function updateBitrate(bitrate) {
  selectedBitrate = bitrate;
  playSong();
}

```

```

function updateSongHistory() {
  fetch('api/history', {
    method: 'GET',
    headers: {
      'Content-Type': 'application/json',
    },
  })
  .then(response => {
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }
    return response.json();
  })
  .then(historyData => {
    const historyList =
document.getElementById('songHistoryList');
    historyList.innerHTML = '';

    const historyArray = Object.entries(historyData);
    const reversedHistory = historyArray.reverse();

    for (let i = 0; i < Math.min(10, reversedHistory.length); i++)
    {
      const [timestamp, song] = reversedHistory[i];
      const listItem = document.createElement('li');
      listItem.textContent = `${timestamp}: ${song.title}`;
      historyList.appendChild(listItem);
    }
  })
  .catch(error => {
    console.error('Error fetching song history:', error);
  });
}

```

Рисунок 3. Приклад js коду з API запитами
Клієнтська логіка для взаємодії з сервером та управління плеєром.

WebSocket:

Отримує повідомлення від сервера про поточний стан (кількість користувачів, команда відтворення).

Оновлює інтерфейс відповідно до отриманих даних.

Функціональність плеєра:

playSong - Відтворює поточну пісню, синхронізуючи час із сервером.

pauseSong та continueSong - Зупиняють і продовжують відтворення відповідно.

updateBitrate - Оновлює бітрейт аудіопотоку.

setVolume - Регулює гучність плеєра.

Історія пісень:

updateSongHistory - Отримує список пісень, які вже грали, і оновлює відповідний блок у DOM.

```
package com.OnlineRadio.OnlineRadioStation.WebSocket;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.socket.config.annotation.EnableWebSocket;
import org.springframework.web.socket.config.annotation.WebSocketConfigurer;
import org.springframework.web.socket.config.annotation.WebSocketHandlerRegistry;

@Configuration
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {

    private final RadioWebSocketHandler radioWebSocketHandler;

    public WebSocketConfig(RadioWebSocketHandler radioWebSocketHandler) {
        this.radioWebSocketHandler = radioWebSocketHandler;
    }

    @Override
    public void registerWebSocketHandlers (WebSocketHandlerRegistry
registry) {
        registry.addHandler(radioWebSocketHandler,
"/ws/radio").setAllowedOrigins("*");
    }
}
```

Рисунок 4. Клас WebSocketConfig

```
package com.OnlineRadio.OnlineRadioStation.WebSocket;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import org.springframework.stereotype.Component;
import org.springframework.web.socket.WebSocketSession;
import org.springframework.web.socket.handler.TextWebSocketHandler;
import org.springframework.web.socket.TextMessage;
import org.springframework.web.socket.CloseStatus;
import java.io.IOException;
import java.util.HashSet;
import java.util.Set;
import java.text.SimpleDateFormat;
import java.util.*;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

@Component
public class RadioWebSocketHandler extends TextWebSocketHandler {

    private static final Set<WebSocketSession> users = new HashSet<>();
    private static final ExecutorService executorService =
Executors.newCachedThreadPool();
}
```

```

    private static final ScheduledExecutorService scheduler =
Executors.newSingleThreadScheduledExecutor();
    private static final Map<String, Integer> userCountHistory = new
LinkedHashMap<>();

    static {

scheduler.scheduleAtFixedRate(RadioWebSocketHandler::collectUserCount, 0,
60, TimeUnit.SECONDS);
    }

    @Override
    public void afterConnectionEstablished(WebSocketSession session)
throws Exception {
        synchronized (users) {
            users.add(session);
        }
        broadcastUserCount();
    }

    @Override
    public void afterConnectionClosed(WebSocketSession session,
CloseStatus status) throws Exception {
        synchronized (users) {
            users.remove(session);
        }
        broadcastUserCount();
    }

    public void sendPlayCommand() {
        String message = "{\"action\":\"playSong\"}";

        synchronized (users) {
            for (WebSocketSession session : users) {
                executorService.submit(() -> {
                    try {
                        if (session.isOpen()) {
                            session.sendMessage(new TextMessage(message));
                        }
                    } catch (IOException e) {
                        System.err.println("Error sending message: " +
e.getMessage());
                    }
                });
            }
        }
    }

    private static void broadcastUserCount() {
        String message = "{\"action\":\"updateUserCount\", \"count\":\" +
users.size() + \"}";

        synchronized (users) {
            for (WebSocketSession session : users) {
                executorService.submit(() -> {
                    try {

```

```

        if (session.isOpen()) {
            session.sendMessage(new TextMessage(message));
        }
    } catch (IOException e) {
        System.err.println("Error sending user count: " +
e.getMessage());
    }
    });
}

private static void collectUserCount() {
    SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss");
    String timestamp = sdf.format(new Date());
    int userCount;
    synchronized (users) {
        userCount = users.size();
    }
    synchronized (userCountHistory) {
        userCountHistory.put(timestamp, userCount);
    }
}

public Map<String, Integer> getUserCountHistory() {
    synchronized (userCountHistory) {
        return new LinkedHashMap<>(userCountHistory);
    }
}
}

```

Рисунок 5. Клас RadioWebSocketHandler

Клас WebSocketConfig забезпечує конфігурацію та інтеграцію WebSocket у додатку.

Клас RadioWebSocketHandler:

Забезпечує обробку підключень клієнтів через WebSocket.

Дозволяє відправляти повідомлення клієнтам у реальному часі (наприклад, команду відтворення музики або оновлення кількості користувачів).

Збирає статистику про кількість підключених користувачів для подальшого аналізу.

Висновок: На цій лаб. роботі я зрозумів суть різних видів взаємодії додатків, а саме CLIENT-SERVER, PEER-TO-PEER, SERVICE-ORIENTED ARCHITECTURE, реалізував взаємодію CLIENT-SERVER для свого проекту онлайн радіо станції за допомогою Java Spring Boot та технології web socket, що допомогла створити справжнє радіо з відтворенням музики у реальному часі.